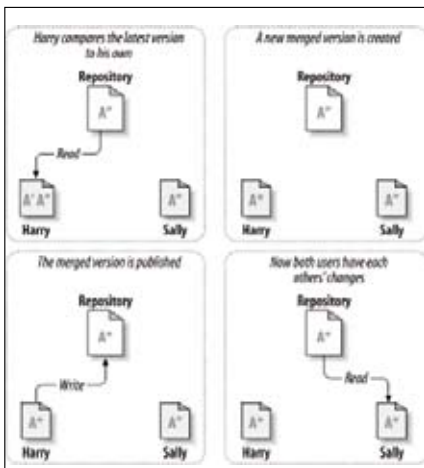




After modifications are made, the changes can be merged

When BetaDood tries to update his copy, he will be told that his copy is out of date, since his updates were made to revision 1, while the current revision is 2.

BetaDood will now have to first merge the changes in revision 2 into his local copy. This will be a simple process, as changes will usually be made in different parts of the document and will not conflict. His latest copy, now based on revision 2 can now be merged into the main file, giving us revision 3.

This may seem like a tedious process but there is rather little effort involved in most common scenarios, and this ends up being a much more efficient and element solution.

Change again is the most important factor here and that is what drives any revision control software.

Quantifying change

A variety of systems exist for maintaining revisions. However, there is one thing they all have in common. They include a mechanism to detect the changes made to files.

Using revision control you have access to every change ever made that was submitted or committed to the repository. Like a time machine that can stop at any and every revision ever made in any file!

It may seem like storing each and every version of each and every file will take a rather large amount of space, but however, usually, revision control programs are very optimised for changes in files, since that is their primary metric anyway.

Revision control softwares use smart algorithms to store the minimal amount of data needed to differentiate each revision from the previous one. Only the changes made in each revision are actually stored.

If a copy of the repository contents is made either for release (tagging)

or for starting an offshoot / spin-off of the project (branching) or experimenting with a new feature (branching), the system usually creates only an internal reference to the revision it is associated with, instead of an actual physical copy.

When working with files in a revision controlled system, you will often come across trunks and branches, tags and tips – it's time for a biology lesson.

Of trunks and branches

In the programming world, branching and tagging are concepts that pop up frequently. For any software project, the "trunk" of the code is that which contains the latest revision of the code made to date. It is the one shooting upwards towards the next version. Branches in a code occur when a developer decides to take a different approach to the code thereby "branching" out in a different direction; maybe a different UI, or a different feature, or an experimental approach to a problem. These changes are often merged back into the original trunk when they are considered stable enough.

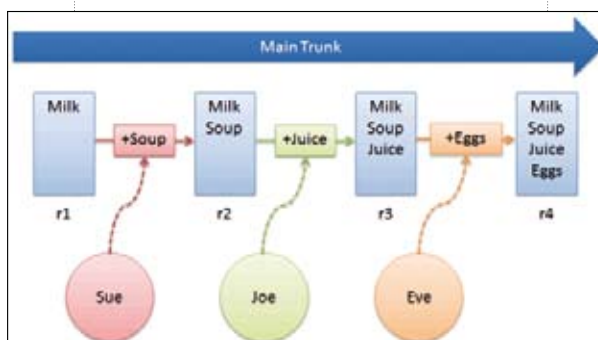
Take for example an evolving work of art done in Paint, a silly concept, but humour us. As you make each significant change, you save and "commit" it to the trunk. At one point though we decide that the background is too dull, and decide to try a different colour, and here, we make a new branch. When you have a piece that you think stands on its own, or something that is otherwise special, you can tag it.

For storing your files in a revision control system, branches and trunks may not seem as useful, but still it is nice to know they are always there to lend support.

How and where a revision control system manages its content is also an important factor, and based on this concept we have centralised and distributed revision control systems.

Models of Revision

While the centralised system is based on the old client-server model, decentralised revision control systems are built on the increasingly popular P2P model.



The centralised system

Subversion

After reading about the power of Revision control systems, you might be excited to set up your own. For some systems this can be as easy as simply installing a single software tool!

Subversion (SVN) uses a Client Server system for revision control, and as such you will need to set up both a client and server if you intend to use it. If you need it simply for your documents which you will probably work on alone, this can be quite an overkill.

To get Subversion up and running on your computer, you need to install SVN, following which you will need to set up the subversion server. Subversion can work with Apache server, using a module provided with the SVN setup. Unless you have Apache already installed and running, however, it is much easier to set up and configure it's own server. Apache provides some more facilities such as WebDAV support, auto-versioning and a web interface of you repository contents.

Once the server is set up, you can access it using the computers local IP (127.0.0.1) or "localhost" (if you are using it from the same computer).

With SVN you are left always shuffling files back and forth from a server, which can be quite slow, even one which is running on your own computer. Also the server will need to be running if you wish to access the repository, this basically means that the server process will always be running on your computer hogging up memory.

If you intend to use a revisioning system alone on your computer, Mercurial might turn out to be a better option.

Decentralised version control systems have only recently started to become popular, and some of the best known decentralised solutions are just a couple of years old.

Centralised Revisioning

In the client-server model, the code is stored and maintained in a central repository. People send changes directly to this server, and the server alone maintains the histories and change logs of the files.

This concept is quite easy to grasp, as it is one we are quite familiar with already. If you take revisioning out of the equation it is basically like having an FTP server.

In a centralised system, the repository is hosted on a server, which all the team members access in order to access the latest revision of the code, or to submit changes.

It is clear to see where such a system would be advantageous. As all the files are stored in one convenient location, it is easier to control and moderate. It is easier to track changes